Lecture 1 : Introduction à l'Edge computing

Du cloud au TinyML, projet EcoTri 4.0

Pourquoi ne pas envoyer toutes les images au cloud ?

Pr. Aissam BERRAHOU

Département Ingénierie des systèmes embarqués et mobiles, Smart Systems Lab
IA en Edge Computing, projet fédérateur EcoTri 4.0, version 2027
ENSIAS, Université Mohammed V de Rabat

aissam.berrahou@ensias.um5.ac.ma

Feuille de route

- **Pourquoi l'Edge ?** : *Qu'est-ce qui ne va pas avec « tout envoyer au cloud » ?*
- **Du cloud au TinyML** : *Où peut-on placer le calcul, et avec quelles ressources ?*
- **Budget de placement** : *Comment chiffrer latence, débit, énergie et confidentialité ?*
- **Anatomie d'un MCU** : *Que contiennent nos trois cartes, et qu'est-ce que cela permet ?*
- **Méthodologie et synthèse** : *Comment passer du besoin au cahier des charges (jalon J1) ?*

Objectifs

Ce que vous saurez faire en sortant de la salle

À la fin de cette lecture, vous serez capables de :

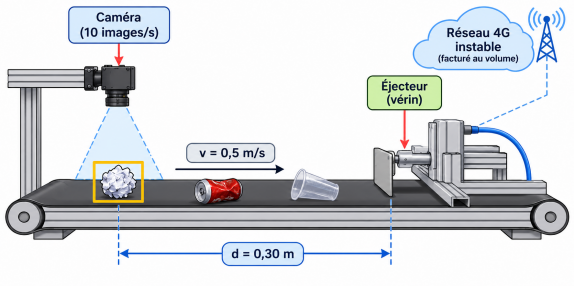
- **Expliquer** pourquoi une station de tri ne peut pas tout confier au cloud, et situer chaque traitement entre cloud, fog, edge et TinyML.
- **Chiffrer** un budget de latence, de débit et d'énergie avec de vrais nombres, plutôt qu'avec des impressions.
- **Lire** la fiche d'un microcontrôleur et dire où vont l'image, les poids du modèle et ses activations (Flash, SRAM, PSRAM).
- **Comparer** nos trois cartes : Portenta H7, XIAO ESP32-S3 Sense et ESP32-CAM.
- **Rédiger** des exigences que l'on peut vérifier, puis dessiner une architecture dont chaque choix se justifie.

Le fil rouge et le livrable

- Nous construisons ensemble, lecture après lecture, la station de tri **EcoTri 4.0**.
- En fin de lecture, votre équipe rend le **jalon J1** : un cahier des charges chiffré, un schéma d'architecture argumenté et le tableau de vos premières mesures.

Le problème déclencheur

Une station de tri, un tapis, 600 ms pour décider



La situation

- Les déchets défilent sur un **tapis** à 0,5 m/s, filmés à **10 images/s**.
- 30 cm plus loin, un éjecteur doit pousser chaque objet vers le bon bac.
- Le site n'a qu'une 4G **instable**, facturée au volume.

À vous (5 min, par équipe)

Pourquoi ne pas tout envoyer à un serveur cloud qui ferait la classification ? Trouvez **trois** raisons, et pour chacune une **grandeur mesurable** (ms, Mbit/s, mJ…).

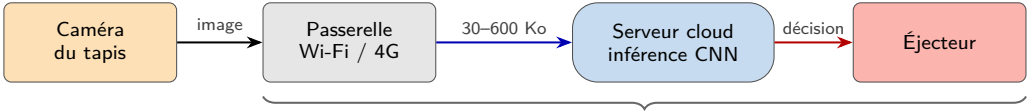
Pourquoi l'Edge ?

Qu'est-ce qui ne va pas avec « tout envoyer au cloud » ?

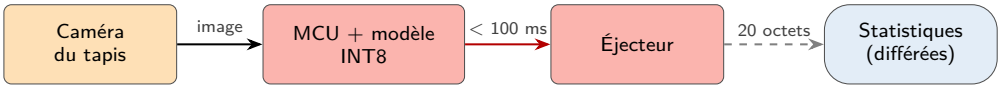
Pourquoi l'Edge ?

Le modèle « tout cloud » et ses limites

(a) Tout dans le cloud



(b) Inférence sur le nœud (edge, TinyML)



traiter la donnée **là où elle naît** et n'envoyer que la **décision** (quelques octets), pas l'image (des centaines de Ko).

Pourquoi l'Edge ?

Les cinq moteurs de l'Edge computing (Shi et al., 2016 ; Satyanarayanan, 2017)

Moteur	Constat	Dans EcoTri 4.0
Latence	le temps aller-retour réseau (RTT) est variable et non borné	l'éjecteur a 600 ms, pire cas compris
Bande passante	les capteurs produisent plus de données que le réseau n'en transporte	49 Mbit/s en VGA brut contre 1 Mbit/s en BLE
Énergie	émettre un bit coûte bien plus cher que le calculer localement	nœud XIAO alimenté sur batterie
Confidentialité	la donnée brute (images, sons) peut identifier des personnes	opérateurs filmés sur le tapis
Fiabilité	le service doit fonctionner hors connexion	4G instable : la station ne doit pas s'arrêter



on peut les mémoriser avec l'acronyme anglais **BLERP** : *Bandwidth, Latency, Economics, Reliability, Privacy* (Warden & Situnayake, TinyML).

Edge computing

Paradigme qui place le **calcul** et le **stockage** à proximité des **sources de données**, en périphérie du réseau, plutôt que dans des centres de données distants.

Edge AI

Exécution de l'**inférence** (et parfois de l'apprentissage) d'un modèle d'IA **sur** l'équipement de périphérie.

TinyML

Sous-domaine de l'Edge AI : apprentissage automatique sur des **microcontrôleurs**, avec une consommation de l'ordre du **milliwatt** et une mémoire de l'ordre de **centaines de Ko**.

Attention

Edge **ne veut pas dire** « sans cloud ». Le cloud reste utile pour **entraîner** les modèles, **agréger** les statistiques et **mettre à jour** les nœuds.

Question flash

?

Une caméra de sécurité envoie un flux vidéo complet au cloud pour y détecter des intrusions. Citez deux moteurs de l'Edge qui plaident pour détecter localement et n'envoyer qu'une alerte.

Question flash

? Une caméra de sécurité envoie un flux vidéo complet au cloud pour y détecter des intrusions. Citez deux moteurs de l'Edge qui plaident pour détecter localement et n'envoyer qu'une alerte.

Réponse

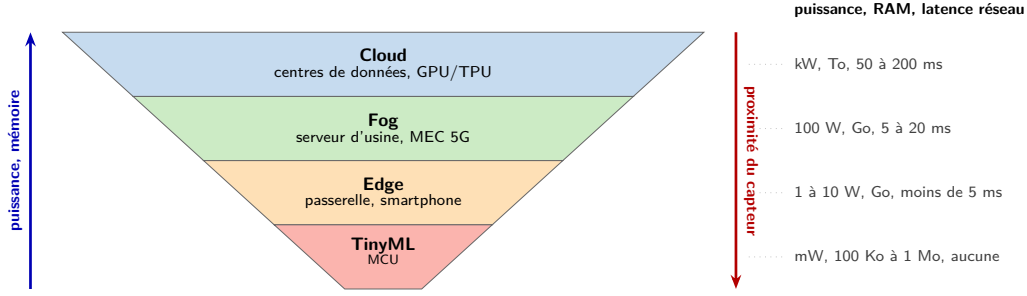
Bande passante (un flux continu sature le lien montant) et **confidentialité** (les images de personnes restent sur place). On peut aussi citer la **fiabilité** : la détection continue même si Internet est coupé.

Du cloud au TinyML

Où peut-on placer le calcul, et avec quelles ressources ?

Du cloud au TinyML

Quatre niveaux, des ordres de grandeur très différents



Plus on descend, plus on se **rapproche du capteur** et plus les **ressources diminuent** : environ 6 ordres de grandeur en puissance et en mémoire entre le haut et le bas.

Cloud

- Ressources quasi **illimitées** : GPU, TPU, To de stockage.
- Idéal pour l'**entraînement**, l'agrégation et l'historique.
- Latence de 50 à 200 ms, **non garantie**.
- Coût récurrent (calcul + transfert de données).

Fog (Cisco, 2012) / MEC

- Couche **intermédiaire** : serveur d'usine, routeur industriel, antenne 5G (*Multi-access Edge Computing*).
- Agrège plusieurs nœuds d'un même site.
- Latence 5 à 20 ms ; disponible **sur le réseau local**.
- Dans EcoTri : la Portenta joue partiellement ce rôle de **passerelle**.

Edge (« riche »)

- Processeurs applicatifs : Raspberry Pi, Jetson, smartphone.
- Système d'exploitation (Linux, Android), Go de RAM.
- 1 à 10 W ; frameworks complets (PyTorch, TFLite).

TinyML (« profond »)

- **Microcontrôleurs** : Cortex-M, ESP32, RISC-V.
- Pas d'OS (ou un RTOS), 100 Ko à quelques Mo de RAM.
- 1 à 500 mW : fonctionne sur **pile** pendant des mois.
- Moteurs dédiés : TFLite Micro, CMSIS-NN, Edge Impulse EON.



nous travaillons au niveau **TinyML** : nos trois cartes sont des MCU. Toute la difficulté est de faire tenir un modèle utile dans **quelques centaines de Ko**.

Ressource	GPU de centre de données	Smartphone	MCU (Portenta H7)
Mémoire vive	80 Go (HBM)	8 Go	**1 Mo** SRAM
Stockage	To	128 Go	**2 Mo** Flash
Calcul	$\approx 10^{15}$ op/s	$\approx 10^{13}$ op/s	$\approx 10^{9}$ op/s
Puissance	300–700 W	3–5 W	**0,1–0,5 W**

Conséquence

Un MobileNetV2 standard (3,4 M paramètres, $\approx$ 14 Mo en float32) **ne tient pas** dans un MCU. Il faudra réduire, quantifier et choisir l'architecture (lecture 5). Le facteur entre mémoires est de l'ordre de 10^4 **à** 10^5.

Question flash

?

Classer chaque traitement d'EcoTri au niveau le plus pertinent (Cloud, Fog, Edge, TinyML) : (a) décider de l'éjection d'une canette ; (b) ré-entraîner le classifieur avec 10 000 nouvelles images ; (c) calculer le taux de remplissage des bacs du site ; (d) détecter la présence d'un objet devant la XIAO.

Question flash

?

Classer chaque traitement d'EcoTri au niveau le plus pertinent (Cloud, Fog, Edge, TinyML) : (a) décider de l'éjection d'une canette ; (b) ré-entraîner le classifieur avec 10 000 nouvelles images ; (c) calculer le taux de remplissage des bacs du site ; (d) détecter la présence d'un objet devant la XIAO.

Réponse

(a) **TinyML/Edge** (contrainte de 600 ms) · (b) **Cloud** (GPU, données massives) · (c) **Fog** (agrégation à l'échelle du site) · (d) **TinyML** (sur le capteur, à très faible énergie).

Budget de placement

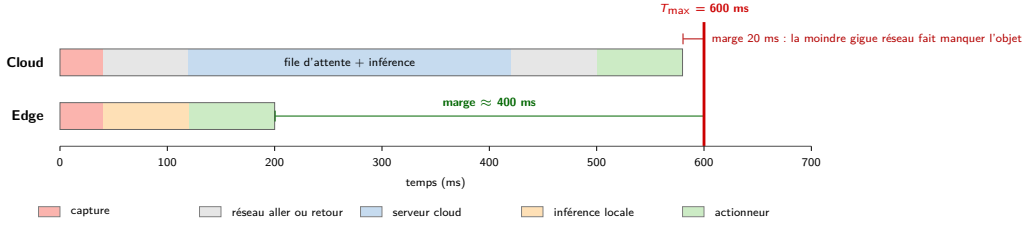
Comment chiffrer latence, débit, énergie et confidentialité ?

- L'échéance est imposée par la **physique** : $T_{\max} = d/v$.
- La chaîne complète compte : capture, prétraitement, inférence, communication, actionnement.
- On raisonne au **pire cas**, pas en moyenne : un objet raté est raté.

Pour EcoTri

$d = 0{,}30$ m et $v = 0{,}5$ m/s, donc $T_{\max} = 600$ ms.

$$T_{\text{capt}} + T_{\text{pré}} + T_{\text{inf}} + T_{\text{com}} + T_{\text{act}} \leq 600 \text{ ms}$$



Interprétation : en moyenne, le cloud passe tout juste (580 ms) ; au moindre pic de gigue 4G, l'objet est déjà passé. L'edge finit en 200 ms et garde 400 ms de marge.

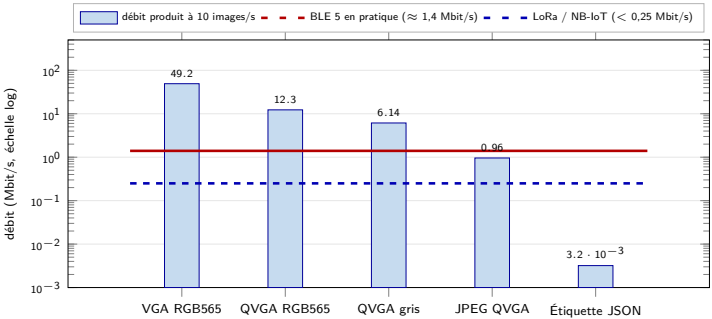
La formule

$$D = W \times H \times b \times f \times 8 \ \text{(bit/s)}$$

$W \times H$: résolution ; b : octets par pixel ; f : images par seconde.

Un exemple : VGA RGB565 à 10 images/s

$640 \times 480 \times 2 \times 10 \times 8 \approx$ **49 Mbit/s**. Une étiquette JSON de 40 octets par objet ne pèse que **3,2 kbit/s**, environ 15 000 fois moins.



Principe : seule l'étiquette passe sous le débit du BLE. Nous suivrons cette règle tout le semestre : **envoyer la décision, pas l'image**.

- Énergie d'une opération : $E = P \times t$ (en J ; 1 mW pendant 1 s = 1 mJ).
- Bilan d'un nœud : $E_{\text{tot}} = E_{\text{capt}} + E_{\text{calc}} + E_{\text{radio}} + E_{\text{veille}}$.
- Ordre de grandeur (Horowitz, ISSCC 2014) : une MAC 8 bits $\approx$ 0,2 pJ ; un accès DRAM 32 bits $\approx$ 640 pJ.
- Transmettre 1 bit par radio coûte typiquement **des centaines à des milliers de fois** plus que le calcul d'une opération.

Exemple : autonomie du nœud XIAO

Batterie 1000 mAh à 3,7 V $\approx$ 13,3 kJ.

Mode « streaming » Wi-Fi permanent ($\approx$ 0,8 W) : $\approx$ **4,6 h**.

Mode « réveil sur détection » (moyenne $\approx$ 40 mW) : $\approx$ **3,8 jours**.

> ! la mémoire et la **radio** dominent la facture énergétique, pas les multiplications.

Confidentialité

- Les images du tapis peuvent contenir des **visages** d'opérateurs : donnée personnelle (loi 09-08 au Maroc, RGPD en Europe).
- Traiter localement = **minimisation** : seule l'étiquette quitte le nœud.

Fiabilité

- Disponibilité attendue de la station : 99,9 % sur 8 h $\Rightarrow$ 29 s d'arrêt toléré par poste.
- Une coupure 4G de 5 min arrêterait une solution « tout cloud ».

Coût d'exploitation (exemple)

- Flux JPEG QVGA à 10 img/s : $\approx$ 0,96 Mbit/s $\approx$ **10 Go par jour** de 24 h.
- Avec un forfait data facturé au Go, la facture devient vite le premier poste de coût.
- Étiquettes JSON : $\approx$ **35 Mo par jour**.

> **!** toujours ramener un argument qualitatif (« c'est plus rapide ») à une **grandeur chiffrée et une hypothèse explicite**.

Budget de placement

Synthèse : la grille de décision

Critère	Question à se poser	Favorise l'Edge si…	EcoTri 4.0
Latence	quelle échéance $T_{\max}$?	$T_{\max} <$ RTT pire cas	600 ms, RTT 4G non borné
Débit	quel volume D produit ?	$D >$ débit du lien	49 Mbit/s $>$ 4G montante
Énergie	sur batterie ?	radio $>$ calcul	nœud XIAO
Confidentialité	donnée personnelle ?	oui	opérateurs filmés
Fiabilité	fonctionnement hors ligne ?	obligatoire	production continue
Ressources	le modèle tient-il ?	oui (après optimisation)	à vérifier (S5)

Le dernier critère est le contre-poids

L'Edge n'est possible que si le modèle tient dans la **mémoire** et le **temps** disponibles : d'où l'anatomie du MCU.

Question flash

?

Une XIAO doit envoyer des images QVGA en niveaux de gris par BLE (débit utile mesuré : 0,7 Mbit/s). Quel est le nombre maximal d'images par seconde ? Que proposez-vous ?

Question flash

? Une XIAO doit envoyer des images QVGA en niveaux de gris par BLE (débit utile mesuré : 0,7 Mbit/s). Quel est le nombre maximal d'images par seconde ? Que proposez-vous ?

Réponse

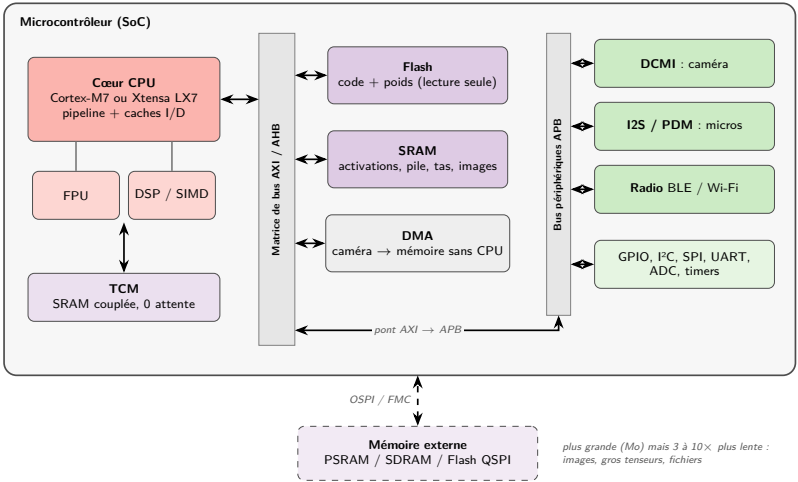
Une image $= 320 \times 240 \times 1$ o $= 76{,}8$ Ko $= 614$ kbit. $0{,}7\,\text{Mbit/s} / 0{,}614\,\text{Mbit} \approx$ **1,1 img/s**. Solutions : n'envoyer qu'à la **détection** d'un objet, **réduire** la résolution (96×96 : 74 kbit, ≈ 9 img/s), compresser en JPEG, ou mieux **classer sur place** et n'envoyer que l'étiquette.

Anatomie d'un MCU

Que contiennent nos trois cartes, et qu'est-ce que cela permet ?

Anatomie d'un MCU

Les blocs d'un microcontrôleur pour l'IA



Anatomie d'un MCU

Les mémoires : rôle et ordres de grandeur

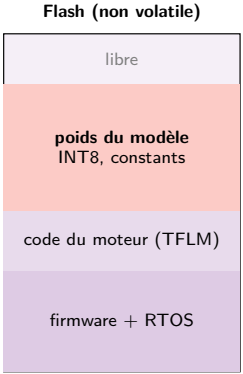
Mémoire	Nature	Taille typique	Vitesse	Contenu en TinyML
Flash	non volatile, lecture seule à l'exécution	1–16 Mo	lente (cache, XIP)	firmware, **poids** du modèle
SRAM	volatile, sur la puce	256 Ko–1 Mo	1 cycle	pile, **activations**, frame buffer
TCM	SRAM couplée au cœur	64–128 Ko	0 attente	boucles critiques, petits tenseurs
PSRAM / SDRAM	externe, volatile	2–8 Mo	3–10× plus lente	images, gros tenseurs
microSD	stockage de masse	Go	très lente	datasets, logs

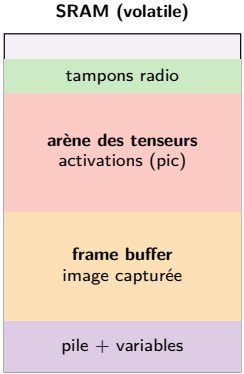


les **poids** sont des constantes : ils vont en **Flash**. Les **activations** changent à chaque inférence : elles vont en **SRAM**. C'est souvent la SRAM qui limite (Lin et al., MCUNet, MIT 2020).

Anatomie d'un MCU

Où va le modèle ? Carte mémoire d'un déploiement





Contraintes de déploiement :

Flash $\geq$ firmware + code + **poids**

SRAM $\geq$ pile + **image** + **pic d'activations** + tampons

*Le **pic** d'activations (et non la somme) dimensionne la SRAM : le moteur réutilise l'arène couche après couche.*

- **FPU** : calcul flottant matériel (simple ou double précision). Sans FPU, un `float` est émulé en logiciel (≈ 10 à $100\times$ plus lent).
- **DSP / SIMD** : une instruction traite plusieurs données. Ex. `SMLAD` (Cortex-M4/M7) : deux MAC 16 bits par cycle.
- **MAC** (*multiply-accumulate*) $a \leftarrow a + w \cdot x$: opération élémentaire d'un réseau de neurones.
- ESP32-S3 : instructions vectorielles 128 bits (PIE) exploitées par ESP-NN.

Ordre de grandeur de la latence

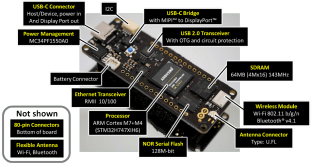
$$t_{\text{inf}} \approx \frac{\text{nombre de MAC}}{\text{MAC par cycle} \times f_{\text{CPU}}}$$

Ex. : 6 M MAC, 2 MAC/cycle effectifs, 480 MHz $\Rightarrow \approx$ **6 ms** (borne basse).

> **!** les entiers 8 bits divisent la mémoire par 4 par rapport au float32 et permettent le SIMD (CMSIS-NN).

Anatomie d'un MCU

Nos trois cartes



Portenta H7 (unité de décision)



XIAO ESP32-S3 Sense (œil du tapis)

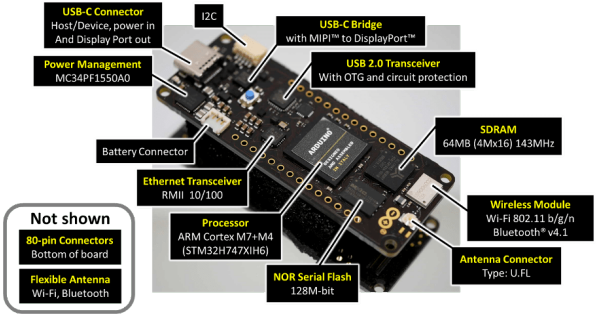


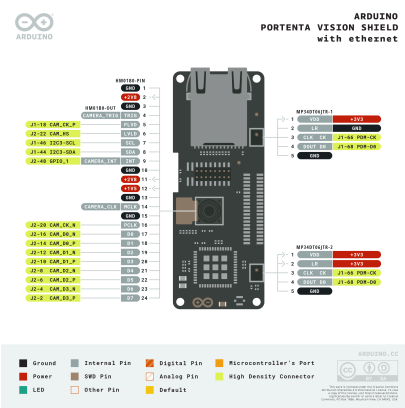
ESP32-CAM AI-Thinker (supervision)

	Portenta H7 + Vision Shield	**XIAO ESP32-S3 Sense**	**ESP32-CAM AI-Thinker**
MCU	STM32H747 : M7 480 MHz + M4 240 MHz	ESP32-S3 : 2× Xtensa LX7 240 MHz	ESP32 : 2× Xtensa LX6 240 MHz
FPU / SIMD	FPU double préc. + DSP	FPU simple + vectoriel 128 bits	FPU simple
SRAM	1 Mo	512 Ko	520 Ko
Mémoire externe	8 Mo SDRAM + 16 Mo QSPI	8 Mo PSRAM + 8 Mo Flash	4 Mo PSRAM + 4 Mo Flash
Capteurs	caméra 320×320 (HM01B0/HM0360), 2 micros	caméra OV2640/OV3660, micro PDM, microSD	caméra OV2640, microSD
Radio	Wi-Fi + BLE 5 (Murata 1DX)	Wi-Fi + BLE 5	Wi-Fi + BT 4.2
Programmation	OpenMV (MicroPython), Arduino	Arduino, ESP-IDF	Arduino, ESP-IDF

Anatomie d'un MCU

La Portenta H7 en détail : retrouver les blocs du schéma





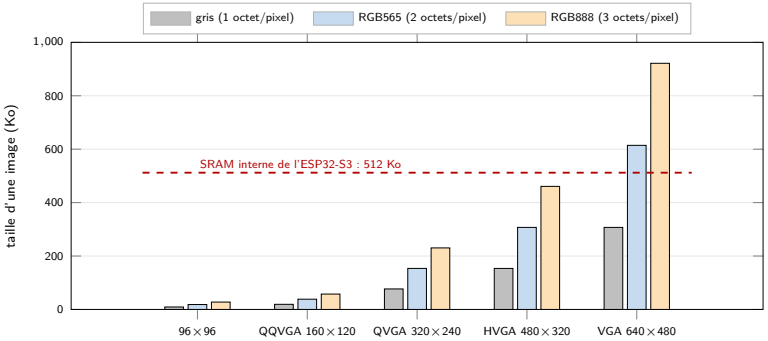
- **Processeur** STM32H747 (M7 + M4) = cœur + SRAM + Flash internes.
- **SDRAM** 64 Mbit et **NOR Flash** QSPI 128 Mbit = mémoire **externe**.
- **Module sans fil** Murata = radio Wi-Fi / BLE.

caméra 320×320 monochrome (interface **DCMI**), 2 micros PDM (**I2S/PDM**), microSD : branché par les connecteurs haute densité 80 broches.

Anatomie d'un MCU

Coût mémoire d'une image : le premier verrou

$$M_{\text{image}} = W \times H \times b$$

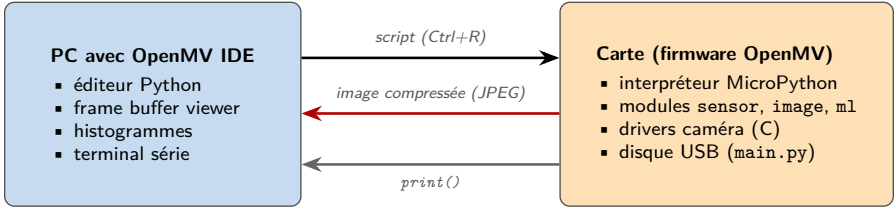


- Une image VGA RGB565 (600 Ko) ne tient pas dans la SRAM de l'ESP32 : elle part en PSRAM.
- Capturer pendant qu'on traite (double tampon) double la note.

! vous mesurerez vous-mêmes FPS et mémoire libre selon la résolution et le format.

Anatomie d'un MCU

Comment OpenMV IDE parle à la carte



USB CDC (câble USB-C). Attention : l'affichage dans l'IDE coûte du temps CPU !

- Le PC envoie le script ; la carte l'exécute avec son interpréteur MicroPython.
- En retour, la carte renvoie l'image compressée et les messages `print()`.
- Cet aller-retour USB a un coût : pour mesurer la vraie cadence, on désactive l'affichage.

```
import sensor, time, gc

sensor.reset()                         # init caméra
sensor.set_pixformat(sensor.GRAYSCALE)
sensor.set_framesize(sensor.QVGA)      # 320x240
sensor.skip_frames(time=2000)          # stabilisation
clock = time.clock()

while True:
    clock.tick()
    img = sensor.snapshot()            # -> frame buffer
    print("FPS %.1f  RAM libre %d o"
          % (clock.fps(), gc.mem_free()))
```

- `snapshot()` dépose l'image dans le **frame buffer**, en dehors du tas MicroPython.
- `gc.mem_free()` indique ce qu'il reste dans le **tas** Python.
- Regardez les FPS avec puis sans l'affichage de l'IDE : la différence surprend souvent.

Question flash

?

Un modèle INT8 a 250 000 paramètres et un pic d'activations de 300 Ko. Il prend en entrée une image 160×160 RGB565. Tient-il sur la XIAO ESP32-S3 (512 Ko SRAM, 8 Mo PSRAM, 8 Mo Flash) ? Où placer chaque élément ?

Question flash

? Un modèle INT8 a 250 000 paramètres et un pic d'activations de 300 Ko. Il prend en entrée une image 160×160 RGB565. Tient-il sur la XIAO ESP32-S3 (512 Ko SRAM, 8 Mo PSRAM, 8 Mo Flash) ? Où placer chaque élément ?

Réponse

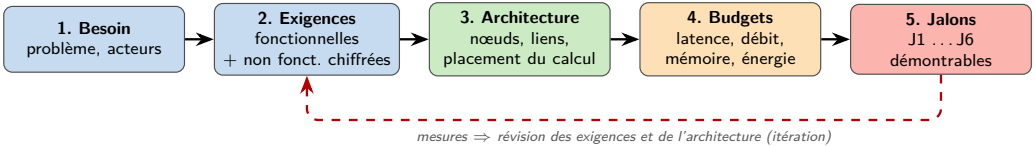
Poids : 250 Ko en **Flash** (large). Image : $160 \times 160 \times 2 = 51$ Ko. Activations : 300 Ko. Total SRAM $\approx$ 351 Ko + pile + Wi-Fi/BLE ($\approx$ 60–100 Ko) : **juste**. Solution sûre : frame buffer en **PSRAM**, arène des tenseurs en SRAM interne (rapide).

Méthodologie et synthèse

Comment passer du besoin au cahier des charges (jalon J1) ?

Méthodologie de projet

De besoin au jalon : une démarche itérative



Exigence fonctionnelle (ce que fait le système)

- EF-1 : la station classe chaque objet en 4 matériaux.
- EF-2 : elle alerte l'opérateur en cas d'anomalie moteur.
- EF-3 : elle compte les objets par lot (QR code).

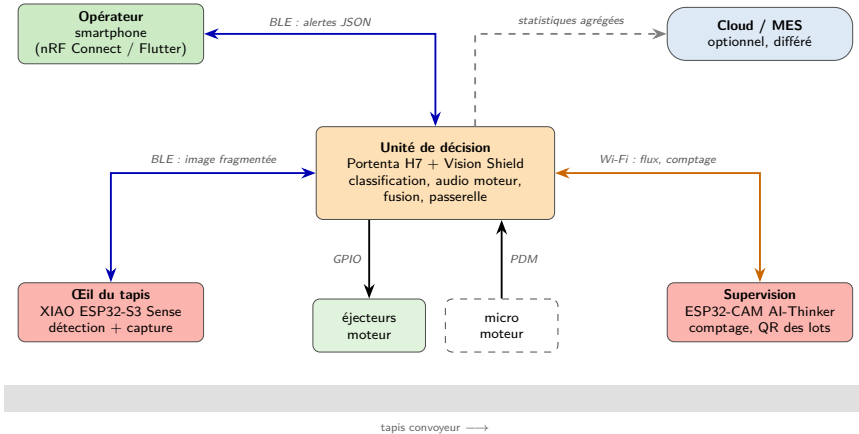
Exigence non fonctionnelle (avec quelle qualité)

- ENF-1 : décision en moins de **600 ms** (pire cas).
- ENF-2 : précision $\geq$ **85 %** sur le jeu de test.
- ENF-3 : aucune image ne quitte le site.
- ENF-4 : nœud XIAO autonome **8 h** sur batterie.

À éviter	À préférer
« Le système doit être rapide. »	« Latence capture→décision $\leq$ 600 ms au 99e centile. »
« Bonne précision. »	« Précision $\geq$ 85 %, rappel métal $\geq$ 90 %. »
« Faible consommation. »	« Courant moyen $\leq$ 40 mA sous 3,7 V. »
« Petit modèle. »	« Poids $\leq$ 1 Mo Flash, pic RAM $\leq$ 300 Ko. »

- Chaque exigence est **numérotée** et associée à une **méthode de vérification** (mesure, test, démonstration).
- Les mesures du TP alimentent la **révision** des exigences : c'est l'itération.

! cahier des charges de l'équipe, **schéma d'architecture** et **tableau de mesures** (FPS, mémoire) des trois cartes.



Chaque flèche porte une **décision de placement** : à justifier par la grille de la section 3.

Méthodologie de projet

Le semestre en sept sous-projets

S	Sous-projet	Théorie dominante	Jalon
1	**Introduction à l'Edge computing**	**Edge computing**	**Cahier des charges + mesures**
2	Voir sans apprendre	Vision embarquée à règles	Référence à règles
3	Faire parler la station	BLE, Wi-Fi, protocoles	Protocole documenté
4	Écouter la machine	TinyML, DSP, données	Dataset + modèle audio
5	Reconnaître le matériau	CNN embarqués, optimisation	Rapport d'optimisation
6	Assembler la station	Déploiement, énergie, fusion	Démonstrateur
7	Industrialiser et défendre	MLOps, sécurité, I4.0	Soutenance + examen

Question flash

?

La direction propose : « Mettons une caméra IP 1080p à 30 img/s et un serveur GPU dans le cloud, c'est plus simple. » Donnez trois contre-arguments **chiffrés** et une situation où cette proposition serait raisonnable.

Question flash

?

La direction propose : « Mettons une caméra IP 1080p à 30 img/s et un serveur GPU dans le cloud, c'est plus simple. » Donnez trois contre-arguments **chiffrés** et une situation où cette proposition serait raisonnable.

Réponse

1080p brut RGB565 à 30 img/s $\approx$ 1 Gbit/s (même compressé H.264 : 4–8 Mbit/s, soit 40–85 Go/jour) ; RTT 4G pire cas $>$ 600 ms : objet manqué ; images d'opérateurs hors site (confidentialité). Raisonnable pour une **analyse différée** (audit qualité hors ligne) ou un site avec **fibre** et un serveur Fog local.

À retenir

- **Le continuum** : cloud, fog, edge et TinyML ne s'opposent pas ; on place chaque traitement à l'étage qui respecte ses contraintes.
- **Les moteurs de l'Edge** : latence, bande passante, énergie, confidentialité et fiabilité.
- **Toujours chiffrer** : $T_{\max} = d/v$, $D = W \times H \times b \times f$, $E = P \times t$, avec des hypothèses écrites noir sur blanc.
- **Dans un MCU** : les poids vont en Flash, l'image et les activations en SRAM ; c'est souvent la SRAM qui bloque.
- **Nos cartes** : la Portenta décide (1 Mo de SRAM, OpenMV), la XIAO observe le tapis (8 Mo de PSRAM), l'ESP32-CAM supervise en Wi-Fi à bas coût.
- **La méthode** : des exigences mesurables, une architecture justifiée, des jalons démontrables, et la mesure pour trancher.

Glossaire et références

Terme	Définition courte
RTT	temps aller-retour réseau
MCU	microcontrôleur : CPU + mémoires + périphériques sur une puce
SRAM / PSRAM	RAM interne rapide / RAM externe plus lente
MAC	multiplication-accumulation
INT8	entiers 8 bits (quantification)
FPS	images traitées par seconde
MEC	Multi-access Edge Computing (5G)

- W. Shi, J. Cao, Q. Zhang, Y. Li, L. Xu, « Edge Computing: Vision and Challenges », *IEEE IoT Journal*, 3(5), 2016.
- M. Satyanarayanan (Carnegie Mellon), « The Emergence of Edge Computing », *IEEE Computer*, 50(1), 2017.
- V. Janapa Reddi et al. (Harvard), *Machine Learning Systems*, mlsysbook.ai ; cours CS249r et edX « Tiny Machine Learning ».
- P. Warden, D. Situnayake, *TinyML*, O'Reilly, 2019.
- J. Lin, W.-M. Chen, . . . , S. Han (MIT), « MCUNet: Tiny Deep Learning on IoT Devices », NeurIPS 2020.
- C. Banbury et al., « MLPerf Tiny Benchmark », NeurIPS Datasets & Benchmarks, 2021.
- M. Horowitz (Stanford), « Computing's Energy Problem », ISSCC 2014.
- Documentations : Arduino Portenta H7, Seeed XIAO ESP32-S3, Espressif ESP32, OpenMV.

Prochaine étape

Tutorial 1 : prise en main de la Portenta H7 avec OpenMV. **TP 1** : mesures sur les trois cartes et jalon J1.